

05506002 Methods of Numerical Computation

Lab 5

Objectives

1. Develop linear regression method to perform curve fitting for sets of discrete data.
2. Develop different types of polynomial interpolations for fitting the curve with a few numbers of discrete data.

Linear regression

Linear regression is a curve fitting method to estimate a linear line from a set of discrete data that might have some errors.

In Google Colab, create a function as follows to calculate the slope and y-intercept of the linear line using a set of data.

```
import numpy as np
import matplotlib.pyplot as plt

def linear_regression(x, y):

    n = len(x)
    x_mean = np.mean(x)
    y_mean = np.mean(y)

    # Calculate slope (a1)
    numerator = np.sum((x - x_mean) * (y - y_mean))
    denominator = np.sum((x - x_mean) ** 2)
    slope = numerator / denominator

    # Calculate intercept (a0)
    intercept = y_mean - slope * x_mean

    return slope, intercept
```

From this slope and y-intercept data, we can calculate y data from any given x.

```
def predict(x, slope, intercept):
    return slope * x + intercept
```

We will use the following set of data to test our function.

```
x = np.array([1, 2, 3, 4, 5], dtype=float) # Independent variable
y = np.array([2, 4, 5, 4, 5], dtype=float) # Dependent variable
```

```
slope, intercept = linear_regression(x, y)
```

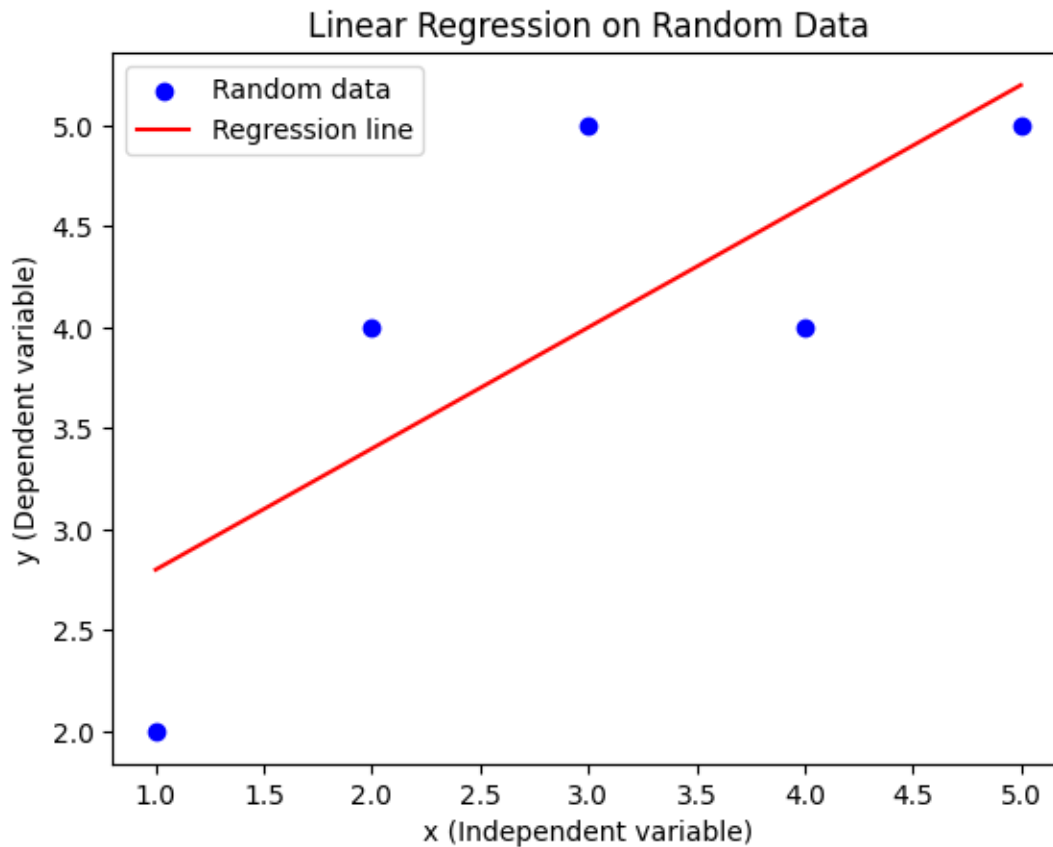
```
y_pred = predict(x, slope, intercept)
```

```
# Print results
print(f"Slope (a1): {slope}")
print(f"Intercept (a0): {intercept}")
print(f"Predicted Y: {y_pred}")
```

```
Slope (a1): 0.6
Intercept (a0): 2.2
Predicted Y: [2.8 3.4 4.  4.6 5.2]
```

We can plot the regression line with the data points as follows.

```
# Plot the results
plt.scatter(x, y, color='blue', label='Random data') # Plot the data
points
plt.plot(np.sort(x), predict(np.sort(x), slope, intercept), color='red',
label='Regression line') # Plot the regression line
plt.xlabel('x (Independent variable)')
plt.ylabel('y (Dependent variable)')
plt.title('Linear Regression on Random Data')
plt.legend()
plt.show()
```



There are two types of error calculation of linear regression which are mean squared error and standard error of estimation. You can see the difference between them in slide week 5. However, you can implement them in Python as follows.

```
def calculate_errors(y, y_pred):
    n = len(y)

    # Residual sum of squares (RSS)
    S_r = np.sum((y - y_pred) ** 2)

    # Mean Squared Error (MSE)
    mse = S_r / n

    # Standard Error of the Estimate (SEE)
    see = np.sqrt(S_r / (n - 2))

    return mse, see
```

```
mse, see = calculate_errors(y, y_pred)

# Display Results
print(f"Mean Squared Error (MSE): {mse:.4f}")
print(f"Standard Error of the Estimate (SEE): {see:.4f}")

Mean Squared Error (MSE): 0.4800
Standard Error of the Estimate (SEE): 0.8944
```

Polynomial Interpolation

Interpolation is a method used to estimate unknown values that fall within the range of known values. Essentially, it involves constructing new data points within the bounds of a discrete set of known data points. This technique is widely applied in various fields such as mathematics, engineering, and data analysis to predict trends, fill in missing data, and create smoother transitions between data points.

Polynomial interpolation, a specific type of interpolation, involves fitting a polynomial to a set of data points. The goal is to find a single polynomial equation whose graph passes through each of the given data points exactly.

We can start with two known points which will create a linear line. This is called linear interpolation which could be implemented in Python as follows.

```
def linear_interpolation(x0, y0, x1, y1, x):
    return y0 + ((y1 - y0) / (x1 - x0)) * (x - x0)
```

We will test with the example 5.1 in slide week 5. We will calculate $\ln 2$ using $\ln 1$ and $\ln 6$. We will calculate its true relative error as follows.

```
def calculate_relative_error(true_value, estimated_value):
    return abs((true_value - estimated_value) / true_value) * 100
```

```

true_ln2 = 0.6931472 # True value of ln(2)

# Case 1: Using ln(1) = 0 and ln(6) = 1.791759
x1, y1 = 1, 0          # ln(1) = 0
x2, y2 = 6, 1.791759   # ln(6) = 1.791759
x = 2                  # Point to estimate

# Perform linear interpolation for case 1
ln2_case1 = linear_interpolation(x1, y1, x2, y2, x)

# Calculate the relative error for case 1
error_case1 = calculate_relative_error(true_ln2, ln2_case1)

print(f"Case 1 (ln(1) and ln(6)): Approximated ln(2) = {ln2_case1:.6f},
      Relative Error = {error_case1:.2f}%")

```

Case 1 (ln(1) and ln(6)): Approximated ln(2) = 0.358352, Relative Error = 48.30%

If we know three points, we can connect them with a quadratic line which can be used to calculate quadratic interpolation which could be implemented in Python as follows.

```

def quadratic_interpolation(x_vals, y_vals, x):
    x0, x1, x2 = x_vals
    y0, y1, y2 = y_vals

    b1 = (y1 - y0) / (x1 - x0)

    b2 = ((y2 - y1) / (x2 - x1) - b1) / (x2 - x0)

    # Quadratic interpolation
    fx = y0 + b1 * (x - x0) + b2 * (x - x0) * (x - x1)
    return fx

```

We will test with the example 5.2 in slide week 5. We will calculate ln 2 using ln 1, ln 4, and ln 6. We will calculate its true relative error as follows.

```

x_vals = [1, 4, 6]
y_vals = [0, 1.386294, 1.791759]
x = 2
true_ln2 = 0.6931472

# Perform quadratic interpolation
ln2_estimated = quadratic_interpolation(x_vals, y_vals, x)

# Calculate relative error
relative_error = calculate_relative_error(true_ln2, ln2_estimated)

print(f"Estimated ln(2): {ln2_estimated:.6f}")
print(f"True Relative Error: {relative_error:.2f}%")

Estimated ln(2): 0.565844
True Relative Error: 18.37%

```

On the higher order of polynomial interpolations, we can implement them in two ways.

First, Newton polynomial interpolation is a method of constructing polynomials that pass through a given set of points. Unlike traditional polynomial interpolation, which seeks a polynomial that satisfies all data points simultaneously, Newton's method builds the polynomial incrementally.

Newton polynomial interpolation can be implemented as follows.

```

def newton_interpolation(x_vals, y_vals, x):

    # Calculate the divided differences table
    n = len(x_vals)
    table = np.zeros((n, n))
    table[:, 0] = y_vals # First column is y-values

    for j in range(1, n):
        for i in range(n - j):
            table[i, j] = (table[i + 1, j - 1] - table[i, j - 1]) /
(x_vals[i + j] - x_vals[i])

    # Extract the coefficients (first row of the divided differences table)
    coefficients = table[0, :]

```

```

# Compute the Newton interpolating polynomial
result = coefficients[0]
product_terms = 1.0
for i in range(1, n):
    product_terms *= (x - x_vals[i - 1])
    result += coefficients[i] * product_terms

return result

```

Another powerful method for polynomial interpolation is the Lagrange polynomial interpolation. In this method, the main idea is to form the interpolating polynomial as a linear combination of these basis polynomials, each of which is zero at all but one of the given points.

Lagrange polynomial interpolation can be implemented as follows.

```

def lagrange_interpolation(x_vals, y_vals, x):
    n = len(x_vals)
    result = 0

    for i in range(n):
        # Compute the Lagrange basis polynomial L_i(x)
        L_i = 1
        for j in range(n):
            if i != j:
                L_i *= (x - x_vals[j]) / (x_vals[i] - x_vals[j])

        # Add the contribution of f(x_i) * L_i(x)
        result += y_vals[i] * L_i

    return result

```

Tasks (Submit through Google Classroom)

Suppose that you have the information of PM 2.5 in some specific time as follows.

TIME	6 AM	8 AM	10 AM	12 PM	2 PM	4 PM	6 PM
PM 2.5	60	160	100	140	120	130	160

Use Linear Regression, Linear Interpolation, and Quadratic Interpolation to PM 2.5 at 7 AM and 11 AM. For Quadratic Interpolation, you can pick any three data points. Compare the result from three methods and discuss the differences of the predicted values.